

Vertex AI Layout to JSON System

Developer and User Manual

Status	Phase 1 handoff manual
Audience	Developer or operator with no prior project knowledge
Verified	16 September 2026
Repository	https://github.com/jawadengineer12/gcp-vertex-ai
Stable service	https://layout-test-api-6euw7jlffa-uc.a.run.app
Original backend source	3ceb766
Focused fix source	230e505
Active revision	layout-test-api-continue-20260916 (100% traffic)

Scope: Phase 1 operation, maintenance, corpus extension, validation, troubleshooting, and Cloud Run deployment. Phase 2 is excluded.

How to use this manual

This manual is both an onboarding guide and an operating procedure. A first-time developer should complete Sections 2 through 5 before changing data or code. Sections 7 and 8 describe the two sample-maintenance paths. Sections 9 through 12 are the release and incident procedures. Section 14 is a guided exercise that combines the entire workflow.

Reader goal	Start here	Stop when
Run an existing layout locally	Sections 2 through 5	A validated JSON file is written and passes the schema check
Add a normal example	Section 7	Corpus and vector counts match and tests pass
Add a relationship-aware example	Section 8	Special-case normalization and dedicated tests pass
Deploy an approved change	Sections 9 through 11	The candidate and stable URLs both pass the API checks
Investigate a 502	Section 12	The request ID identifies the failing stage and the corrected revision is verified

Before using any command: read the label immediately above it. Commands run either in PowerShell at the repository root, inside the activated Python environment, or in the Google Cloud project. Do not paste placeholder values literally.

1. Purpose and verified behavior

The system converts natural-language page-layout intent into strict `LayoutProject` JSON for a downstream Adobe/InDesign-oriented workflow. It combines deterministic layout planning, template/requirement resolution, retrieval, reranking, Gemini generation, bounded retries, and validation.

“Training” means corpus extension, not Gemini fine-tuning. Adding an example appends an input/expected-JSON pair to the retrieval library and regenerates embeddings. The repository contains no Gemini foundation-model tuning job or tuned-model endpoint.

```
User prompt → deterministic layout plan/template requirements → information collection
→ local vector + BM25 retrieval → CrossEncoder reranking → sanitized context
→ Gemini JSON generation → strict validation
→ bounded retry → local/GCS output and optional publisher handoff
```

Entry point	File	Purpose
CLI	<code>main.py</code>	Local interactive generation; optional <code>--publish</code> .
API	<code>api.py</code>	Stateless FastAPI local/Cloud Run test surface.
Layout Generator	<code>streamlit_app.py</code>	Internal Streamlit client for the API.
Pipeline	<code>services/pipeline_service.py</code>	Shared retrieval, generation, repair, retry, validation.
Schema	<code>schemas/layout_schema.py</code>	Strict converter-facing contract.

1.1 What the system does

The caller chooses one of four templates and describes the desired page or spread. The system resolves required information, finds similar examples, generates strict JSON, repairs deterministic constraint violations where possible, and rejects output that still violates the schema or layout rules. A successful response is suitable for the downstream converter contract. It is not automatically published unless the caller explicitly enables publishing.

1.2 What the system does not do

- It does not fine-tune Gemini when a developer adds examples.
- It does not manage the external publisher dashboard or retrieve a generated PDF.
- It does not deploy a managed Vertex Vector Search index. Retrieval uses the checked-in vector JSON file.
- It does not store API conversation state. A caller must resubmit the original prompt, request ID, and answers after a `needs_input` response. The Streamlit client retains those values in its own session while the page remains open.
- It does not include Phase 2 implementation.

1.3 Processing stages and ownership

Stage	Primary implementation	Observable success
Configuration	<code>core/config.py</code>	Required project, region, paths, and optional output settings load without an exception
Requirement resolution	<code>services/requirement_agent.py</code>	Known values and missing fields match the selected template and deterministic plan
Retrieval and reranking	<code>services/retrieval_service.py</code>	Candidate examples are returned and reranked
Generation	<code>services/generation_service.py</code>	Gemini returns a JSON-compatible object
Validation	<code>services/validation_service.py</code>	Schema and production layout rules pass

Output	<code>services/output_service.py</code>	Validated JSON is returned and optionally saved or handed to the publisher
--------	---	--

2. Access and first-day checklist

2.1 Repository

1. Open `https://github.com/jawadengineer12/gcp-vertex-ai`.
2. Confirm repository `jawadengineer12/gcp-vertex-ai`. It was publicly visible at verification time.
3. Confirm default branch `main`.
4. Clone and enter it:

```
git clone https://github.com/jawadengineer12/gcp-vertex-ai.git
cd gcp-vertex-ai
git branch --show-current
```

Success: `README.md`, `pyproject.toml`, `api.py`, `main.py`, `services/`, `scripts/`, and `tests/` exist. If cloning fails, verify URL, proxy policy, and GitHub access.

The repository was public when this manual was verified. That does not establish whether a named person has been formally onboarded or should retain access if the repository becomes private. The project owner must confirm the GitHub organization, repository owner, and the client account that should administer access. Do not place personal access tokens in `.env` or repository files.

The screenshot displays the GitHub interface for the repository `jawadengineer12/gcp-vertex-ai`. The repository is marked as 'Public'. The main content area shows a list of files and folders under the `main` branch. The folders include `baseline_outputs`, `core`, `docs`, `normalized_data`, `raw_data`, `schemas`, `scripts`, `services`, `special_case_outputs`, `templates`, `tests`, and `utils`. The files listed are `.dockerignore`, `.env.example`, `.gcloudignore`, `.gitignore`, and `.python-version`. The right sidebar provides repository statistics: 0 stars, 0 forks, and 0 watchers. It also includes links to the README, activity, and a report repository button.

Verified repository, default branch, and top-level folders.

2.2 Branch

A branch is a named Git line of work used to isolate and review a change.

```
git switch main
git pull --ff-only
git switch -c feature/add-layout-example
```

Success: `git branch --show-current` prints the new branch.

Work only on the focused branch. Before committing, use `git status --short` to distinguish intended files from local archives, generated output, credentials, or unrelated user changes. A branch is not delivered merely because it exists locally; it must be pushed and reviewed or merged according to the team process.

2.3 Google Cloud

Item	Verified value
Project	angular-lambda-421320
Region/service	us-central1 / layout-test-api
Runtime identity	vertex-layout-dev-sa@angular-lambda-421320.iam.gserviceaccount.com
Inbound key	Secret Manager layout-api-key ; value never documented.
Publisher key	Separate Secret Manager publisher-api-key .

Run in Windows PowerShell:

```
gcloud.cmd auth login
gcloud.cmd config set project angular-lambda-421320
gcloud.cmd auth application-default login
gcloud.cmd config get-value project
```

Success: the last command prints the project and `gcloud.cmd auth list --filter=status:ACTIVE` shows the intended account. Local Python uses ADC; Cloud Run uses its service identity. If access fails, ask the project administrator. The repository verifies `roles/aiplatform.user` for model calls and Secret Manager accessor for mapped runtime secrets; organization-specific deployer roles were not fully enumerated.

Actor	Minimum responsibility	Verification
Local developer	Clone or pull the repository, create ADC, call Vertex in the approved project	<code>gcloud auth list</code> , project check, and a non-publishing generation
Cloud Run deployer	Build source, deploy revisions, update traffic, and read logs	Zero-traffic deployment and service description succeed
Runtime service account	Call Vertex and access only mapped secrets or approved GCS objects	Revision configuration shows the expected identity and the application completes a request
Secret administrator	Create or rotate secret versions without exposing values	Secret metadata and numeric version mapping are correct

The exact human IAM roles depend on the client organization. Typical permissions may be delivered through custom roles or groups, so this manual does not invent role bindings. Ask the project administrator for least-privilege access if any verification command returns Permission Denied.

GCP project and local authentication verification

```
PS D:\Databiqs\GCP Vertex ai> gcloud.cmd config get-value project
angular-lambda-421320

PS> gcloud.cmd auth list --filter=status:ACTIVE
ACTIVE ACCOUNT: [redacted]

PS> gcloud.cmd run services describe layout-test-api --region=us-central1
Service: layout-test-api
Region: us-central1
Runtime service account: vertex-layout-dev-sa@angular-lambda-421320.iam.gserviceaccount.com
Status: Ready

Credential values are intentionally omitted.
```

Redacted project, authentication, service, region, and runtime identity verification.

3. Repository map

Path	Purpose
core/	Configuration, logging, Vertex client.
services/	Primary application implementation.
schemas/	Strict Pydantic layout contract.
templates/	Four template definitions.
raw_data/	Training workbook, font list, source/reference data.
normalized_data/layout_prompt_library_updated.json	Normal retrieval corpus (32 verified examples).
normalized_data/vertex_index_data.json	Checked-in vectors (32 × 768 verified).
normalized_data/special_case_library.json	Three whole-project relationship cases.
scripts/	Import, normalization, embedding, retrieval, and index utilities.
tests/	58 verified tests after the focused retry change.
baseline_outputs/	Four checked-in normal JSON baselines.
special_case_outputs/	Three checked-in special-case JSON outputs.

3.1 Files excluded from deployment or Git

`.gitignore` and `.gcloudignore` serve different purposes. Git exclusions prevent local credentials, environments, logs, and generated files from entering source control. Cloud build exclusions keep unnecessary or sensitive files out of the deployment archive. The current Cloud Run source upload excludes `.git`, environment files, secret directories, key files, virtual environments, caches, logs, generated outputs, documentation, tests, most raw data, archives, and alternate vector-index files. It explicitly retains `raw_data/Font List.csv` and the configured `normalized_data/vertex_index_data.json`.

Checked-in items can still be excluded from the Cloud Run archive. For example, the tracked training workbook remains available to developers but is not required by the API container. Verify exclusions with `gcloud meta list-files-for-upload` before a release. Stop if it prints a credential, `.env`, private key, or unexpected archive.

Storage clarification: both output folders are in the Git repository, not GCP Drive. They are not uploaded to GCS automatically. `GCS_BUCKET` controls optional runtime output only.

4. Local setup

4.1 Virtual environment

Start: PowerShell in the repository root.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python --version
python -c "import sys; print(sys.executable)"
```

Success: prompt normally shows `(.venv)` , Python is 3.12+, and executable is under `.venv\Scripts` . If activation is blocked, use `.\.venv\Scripts\python.exe` explicitly or follow administrator policy; do not weaken machine-wide security settings.

Activation applies only to the current terminal. Open a new terminal only when a procedure explicitly calls for another process, such as starting the API and Streamlit separately. Activate the environment in each terminal, or prefix commands with `uv run` . Do not run maintenance scripts from `scripts/` ; remain at the repository root so relative paths resolve correctly.

4.2 Dependencies

```
uv sync --frozen
uv lock --check
```

Success: both exit 0. The XLSX importer uses pandas and requires an Excel engine available in the environment. Alternative: `python -m pip install -e .` ; prefer the lockfile for reproducible handoff work. If pandas reports a missing `openpyxl` engine, install it in the active environment before running the importer; it is not declared directly by this baseline.

Python virtual environment and dependency setup

PS D:\Databiqs\GCP Vertex ai> python -m venv .venv
PS> .\.venv\Scripts\Activate.ps1
(.venv) PS> python --version
Python 3.12.4

(.venv) PS> uv sync --frozen
Installed project dependencies from uv.lock

Success: the prompt begins with (.venv), Python is 3.12+, and sync exits 0.
If the XLSX importer reports a missing Excel engine, install openpyxl in this environment.

Verified environment, Python, lock synchronization, and XLSX dependency.

4.3 Configuration

```
Copy-Item .env.example .env
```

Set at least `GOOGLE_CLOUD_PROJECT=angular-lambda-421320` and `GOOGLE_CLOUD_LOCATION=us-central1` . Never commit `.env` .

Key	Behavior/default
PROMPT_LIBRARY_PATH	normalized_data/layout_prompt_library_updated.json
SPECIAL_CASE_LIBRARY_PATH	normalized_data/special_case_library.json

EXCEL_FILE_PATH	raw_data/trainingData_Local.xlsx
VERTEX_INDEX_DATA_PATH	normalized_data/vertex_index_data.json
FONT_LIST_PATH	raw_data/Font List.csv
OUTPUT_FILE	outputs/generated_layout.json
LOCAL_OUTPUT	Default true.
GCS_BUCKET / GCS_OBJECT_NAME	Optional GCS destination.
LAYOUT_API_KEY / LAYOUT_API_URL	Inbound API secret and Streamlit target.
PUBLISHER_API_URL / PUBLISHER_API_KEY	Both required for explicit publisher handoff.
ENABLE_RUN_TRACE	Default true locally; traces under outputs/run_traces .
MAX_RETRIES	Default 3 generation attempts.

Configuration precedence: shell environment variables override values loaded from `.env` . Cloud Run uses revision environment variables and Secret Manager mappings rather than the developer's local file. When troubleshooting, verify the environment where the failing process actually runs.

Configuration type	Local development	Cloud Run
GCP identity	Application Default Credentials created by <code>gcloud auth application-default login</code>	Attached service account
Non-secret settings	<code>.env</code> or current PowerShell session	Revision environment variables
Inbound and publisher keys	Current session or uncommitted <code>.env</code>	Pinned Secret Manager versions
Output files	Repository-relative local path or approved GCS bucket	Ephemeral container file system or approved GCS bucket

4.4 First checks

```
python -m unittest tests.test_layout_pipeline.LocalVectorStoreTests
python -m unittest discover tests
python tests/validate_prompt_library.py
```

Verified: 3 vector tests pass; full suite reports 58 tests OK; prompt validation reports 32 examples and 108 Article assets.

5. Run locally

5.1 CLI

```
python main.py
```

Select a template, enter a prompt, answer requested fields, and wait. **Success:** Validated JSON saved (local): ... and the configured output exists. Failure after all retries exits 1 and prints validation details; inspect `logs/pipeline.log` . Publishing is opt-in only:

```
python main.py --publish
```

Use `--publish` only when a real external job is authorized.

5.2 API

Terminal 1:

```
$env:LAYOUT_API_KEY="replace-with-a-long-random-local-value"
$env:ENABLE_RUN_TRACE="false"
uv run uvicorn api:app --host 0.0.0.0 --port 8080 --workers 1
```

Success: Uvicorn listens and `Invoke-RestMethod http://localhost:8080/health` returns `{"status":"ok"}`. Protected routes without the key should return 401.

Fully resolved request:

```
$headers=@{"x-api-key"=$env:LAYOUT_API_KEY}
$body=@{request_id=$null;prompt="Create a one-page article titled Service
Verification";template_id="magazine_article";answers=@{article_title="Service Verification";author_name="Test
Operator";hero_image_url=$null};publish=$false}|ConvertTo-Json -Depth 5
Invoke-RestMethod http://localhost:8080/v1/layouts -Method Post -Headers $headers -ContentType application/json -Body $body
```

Success: HTTP 200, `status=completed`, non-empty `layout`, and `publisher=null`. HTTP 200 `needs_input` is normal: resend the original prompt, returned request ID, and all answers. The API stores no session.

HTTP result	Meaning	Operator action
200 <code>needs_input</code>	The request is valid but required template fields are unresolved	Enter the fields and select Submit details ; the Streamlit client resubmits the same request ID
200 <code>completed</code>	Generation and validation passed	Inspect or download the layout; confirm publisher is null when publishing was disabled
502 <code>generation_failed</code> , <code>retryable=true</code>	Generation exhausted its configured validation attempts	The Layout Generator shows Continue ; selecting it starts a fresh request with the saved prompt, template, answers, and publish choice
502 <code>publish_failed</code>	Validated layout exists but outbound handoff failed	Preserve the layout and request ID; check publisher state before retrying because a duplicate job is possible
401	Inbound API key missing or incorrect	Load the current secret without printing it
422	Request body, template, answer, or URL violates the API contract	Read the structured detail and correct the request
503	Server or publisher configuration is unavailable	Correct configuration; the retry Continue control is not shown for this response

5.3 Layout Generator

Terminal 2:

```
$env:LAYOUT_API_URL="http://localhost:8080"
uv run streamlit run streamlit_app.py
```

Open `http://localhost:8501`, enter the API URL and same password-masked key, select Connect, choose a template, enter a prompt, keep Publish off, and submit. **Success:** Connected, four templates, "Layout generated successfully. It was not published.", timings, validated JSON, and Download JSON.

5.3.1 Missing information

1. Choose Magazine Article.

2. Enter a prompt that omits its title or author.
3. Select Generate layout.
4. Expect HTTP 200 `needs_input` . The interface displays A few more details. This is expected, not an error.
5. Enter the requested values and select **Submit details**. This control is intentionally not labeled Continue.

Success: generation begins using the same request ID and accumulated answers. If submission returns 422, inspect the answer type and require HTTPS for image URLs.

5.3.2 Continue after exhausted generation attempts

1. If Gemini cannot produce a valid layout after the configured attempts, the API returns HTTP 502 with `status=generation_failed` , the failed request ID, and `retryable=true` .
2. The Layout Generator preserves the prompt, selected template, accumulated answers, and publish choice in its session.
3. The page displays: “This layout is taking longer than expected and could not be completed after the available attempts.”
4. Select **Continue**.
5. The client sends the saved input again with `request_id=null` . The API creates a fresh request ID and runs the complete pipeline again.

Success: Continue is absent before a retryable generation failure; it appears after the structured 502 response; the fresh request completes; the retry panel disappears; validated JSON and Download JSON are available. Closing or refreshing the Streamlit page may discard the locally retained input. Publisher failures do not show Continue because blind resubmission could create a duplicate external job.

Deploy ⋮

API connection

API URL

http://127.0.0.1:8765

API key

Connect

Layout Generator

Generate and validate an InDesign layout through the existing API.

Template

Magazine Article

Describe your layout

Create a magazine article about resilient cities.

☐ Publish result ⓘ

Generate layout

This layout is taking longer than expected and could not be completed after the available attempts.

Select Continue to start a fresh generation request with the same prompt, template, answers, and publishing choice.

Continue

Deterministic verification of the retry screen after an exhausted generation response. Continue appears only after the structured 502 failure.

API connection

API URL

`http://127.0.0.1:8765`

API key



Connect

Layout Generator

Generate and validate an InDesign layout through the existing API.

Template

Magazine Article



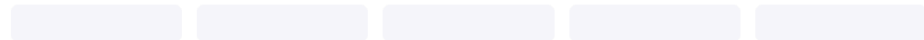
Describe your layout

Create a magazine article about resilient cities.

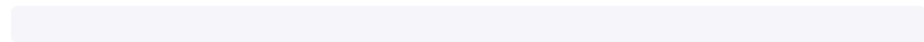
☐ Publish result ⓘ

Generate layout

Layout generated successfully. It was not published.

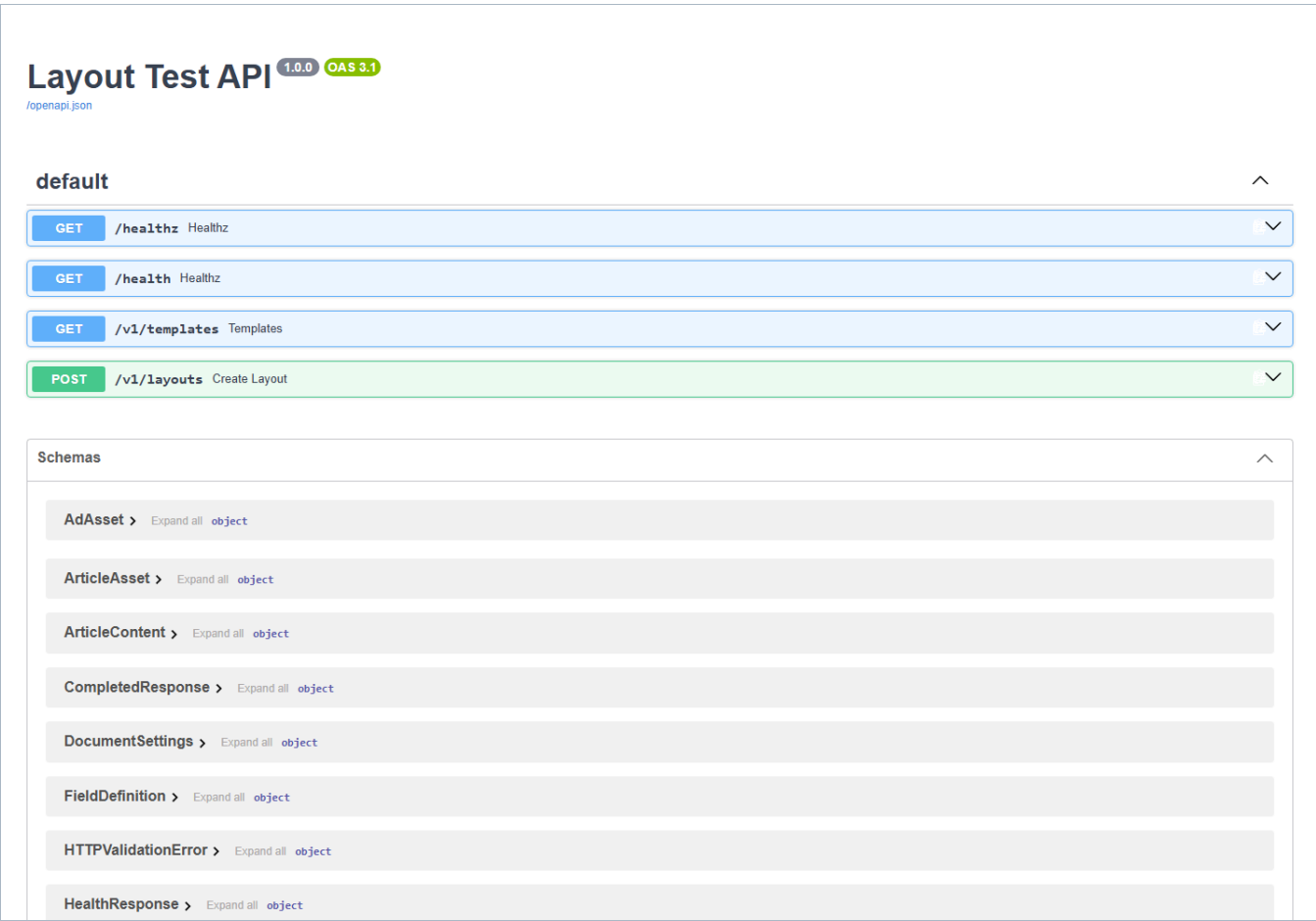


Validated layout JSON



Start over

The fresh request completed with the saved prompt and answers. The retry panel disappeared and publishing remained disabled.



Deployed Swagger page showing health, templates, and layout routes.

6. Templates, special cases, and validation

The source of truth for the four templates (`magazine_article`, `editorial_page`, `cover_page`, `advertisement`) is `templates/*.json` and protected `GET /v1/templates`. Deterministic layout planning may add prompt-specific image requirements.

A **special case** is a whole layout whose meaning depends on relationships: same-page threaded text, cross-page threaded text, or cross-page image spread. Source: `normalized_data/special_case_library.json`; outputs: `special_case_outputs/`; tests: `tests/test_special_cases/`.

Validation covers strict schema/types, page indexes, safe margins/bleed, approved fonts, Article overlap, threading links and `autoFit`, image spreads, page counts, and explicit position/size/column constraints.

```
python -c "import json; from pathlib import Path; from schemas.layout_schema import LayoutProject;
p=Path('baseline_outputs/normal_cover.json'); LayoutProject.model_validate(json.loads(p.read_text(encoding='utf-8')));
print('VALID:',p)"
```

Success: prints VALID and exits 0. This checks schema; runtime `ValidationService` additionally applies layout rules. API `status=completed` and CLI validated-save text are the clearest end-to-end success criteria.

Validation layer	What it proves	What failure looks like
Pydantic schema	Required fields, strict types, enum values, nested assets, and forbidden extra fields are correct	The command exits nonzero with the failing field path

Production validation service	Page bounds, margins, overlap, fonts, threading, spreads, page count, and prompt constraints are safe	The pipeline records validation errors and retries with cumulative feedback
API completion	The shared pipeline produced a schema-valid and rule-valid layout	Status is not <code>completed</code> , or the layout is missing
Baseline regression	Checked-in representative outputs remain accepted by the current schema and rules	A baseline or special output fails its test or validation command

Do not treat valid JSON syntax alone as success. A file can parse with `json.loads` and still violate the converter schema, use an unapproved font, overlap Article frames, place an asset outside page bounds, or break a text thread.

Automated tests and JSON validation

```
(.venv) PS> python -m unittest discover -s tests
Ran 58 tests in 0.480s
OK

(.venv) PS> python tests/validate_prompt_library.py
Validated 32 examples and 108 Article assets.

Schema checks:
VALID baseline_outputs/normal_advertisement.json
VALID baseline_outputs/normal_cover.json
VALID baseline_outputs/normal_editorial.json
VALID baseline_outputs/normal_magazine_article.json
VALID special_case_outputs/cross_page_threaded_text.json
VALID special_case_outputs/image_spread.json
VALID special_case_outputs/same_page_threaded_text.json
```

Verified 74-test suite, prompt validation, and checked-in JSON baselines/special outputs.

7. Normal sample workflow

Use a workbook containing only rows to append. The importer appends every valid non-empty row and does not deduplicate. Run once and inspect the diff.

7.1 Prepare and append

1. Start on a branch.
2. Create `raw_data/new_examples.xlsx`.
3. Include exact columns `Page Number`, `Generated Prompt (Input)`, and `Expected JSON Object (Output)`. The expected value must parse as JSON and contain a non-empty `assets` array.
4. Point the session to it and append:

```
$env:EXCEL_FILE_PATH="raw_data/new_examples.xlsx"
python -m scripts.append_excel_to_prompt_library
```

Success: Appended count equals intended rows, no unexpected invalid/empty rows, and total rises by that count. The verified one-row exercise increased 32 to 33. File/path failures mean the environment variable or working directory is wrong; Excel-engine errors mean dependencies are stale.

The checked-in source workbook is `raw_data/trainingData_Local.xlsx`. Verification found 92 rows and these six columns:

Column	Use
--------	-----

Project Name	Human-readable source grouping
Source JSON File	Reference to the originating expected JSON
Source PDF File	Reference to the originating PDF when available
Page Number	Source page or page context
Generated Prompt (Input)	Retrieval intent stored as <code>intent</code>
Expected JSON Object (Output)	JSON object stored as <code>expected_output</code>

The importer requires the exact workbook headers, including the parenthetical Input and Output text. Do not use the full 92-row workbook as an append source unless every row is intentionally new. The script appends valid rows and does not deduplicate them.

trainingData_Local.xlsx - representative rows

Project Name	Source JSON File	Source PDF File	Page Number	Generated Prompt (Input)	Expected JSON Object (Output)
CCL 66	CCL 66_data.json	CCL 66_Page_1.pdf	1	Create a layout with two diagonal gray rectangles intersecting at the center,	[Expected JSON object - truncated]
CCL 66	CCL 66_data.json	CCL 66_Page_2.pdf	2	Layout: Four gray rectangles forming a cross, each occupying a quadrant of the	[Expected JSON object - truncated]
CCL 66	CCL 66_data.json	CCL 66_Page_3.pdf	3	Create a layout with a bold "CONTENTS" title at top center. Below, arrange four	[Expected JSON object - truncated]

Representative rows from the tracked XLSX workbook. Expected JSON cells are deliberately truncated for readability.

Verified sample and output locations

```
raw_data/trainingData_Local.xlsx
Rows: 92
Columns:
  Project Name
  Source JSON File
  Source PDF File
  Page Number
  Generated Prompt (Input)
  Expected JSON Object (Output)

normalized_data/layout_prompt_library_updated.json  32 examples
normalized_data/vertex_index_data.json              32 vectors x 768 dimensions
normalized_data/special_case_library.json           3 special cases
baseline_outputs/                                   4 normal JSON baselines
special_case_outputs/                              3 relationship-aware JSON outputs
```

Verified workbook columns and all sample/vector/output locations.

7.2 Validate and normalize

```
python tests/validate_prompt_library.py
python -m scripts.normalize_prompt_library `
  --source normalized_data/layout_prompt_library_updated.json `
  --output normalized_data/layout_prompt_library_sanitized.json
```

Runtime retrieval already sanitizes selected examples. The normalization command creates a review artifact; it does not update configured vectors unless configuration is deliberately changed.

One-example append and normalization verification

```
(.venv) PS> python -m scripts.append_excel_to_prompt_library
Loaded existing library | records=32
Loaded Excel | rows=1
Append complete | appended=1 | invalid=0 | empty=0 | total=33
Library updated: ../workflow_test/library_one.json

(.venv) PS> python tests/validate_prompt_library.py
Validated 33 examples and 123 Article assets.

(.venv) PS> python -m scripts.normalize_prompt_library --source ... --output ...
Wrote 33 sanitized examples to ../sanitized.json

Success: exactly one record was appended and validation completed without traceback.
```

Verified one-row append, validation, and normal-library normalization in an isolated copy.

7.3 Regenerate embeddings

```
python -m scripts.prepare_vertex_vector_data
python -m unittest tests.test_layout_pipeline.LocalVectorStoreTests
```

The first command is a paid Vertex `text-embedding-004` call over the full configured library. It rewrites `VERTEX_INDEX_DATA_PATH`.

Success: vector count equals library count, command prints Success, and 3 alignment/dimension tests pass. Commit the prompt library and vector file together.

Embedding regeneration and alignment verification

```
(.venv) PS> python -m scripts.prepare_vertex_vector_data
Loaded library | records=33
Generating embeddings | model=text-embedding-004 | count=33
Vector JSON written | records=33
Success! 33 records written to ../vector_one.json

(.venv) PS> python -m unittest tests.test_layout_pipeline.LocalVectorStoreTests
Ran 3 tests in 0.210s
OK

Success: library and vector counts match, every stable ID resolves, and vectors are 768-D.
```

Verified embedding regeneration to a temporary file and vector alignment.

8. Special-case workflow

1. Document the relationship and evidence.
2. Add/update a whole-project case in a source JSON matching the special library shape.
3. Normalize to a candidate file:

```
python -m scripts.normalize_special_case_library `
--source path/to/special_cases_source.json `
--output normalized_data/special_case_library.candidate.json
```

4. Diff candidate against `normalized_data/special_case_library.json`.

5. Replace only after review and add/update dedicated tests.
6. Run:

```
python -m unittest tests.test_special_cases.test_special_cases
python -m unittest discover tests
```

Success: both end in OK. Special-case changes do not require normal-library embeddings unless the normal prompt library also changed.

9. Representative generation and release checks

The baseline repository has no `demo_readiness_matrix` or `demo_generation_matrix` module. Do not document or run those names on this branch. Use the checked-in deterministic suites first:

```
python -m unittest tests.test_special_cases.test_special_cases
python -m unittest discover -s tests
```

Success: the special-case suite and full suite report OK. For a live representative generation, run `python main.py`, choose the relevant template, use a reviewed prompt, and keep publishing disabled. A live success requires the validated-save message and a saved JSON that passes the Section 6 schema check. Vertex/Gemini calls may cost money and vary between runs.

```
uv lock --check
python -m unittest discover tests
python tests/validate_prompt_library.py
python -m unittest tests.test_layout_pipeline.LocalVectorStoreTests
git diff --check
```

10. Local, GCS, and publisher output

10.1 Local

```
LOCAL_OUTPUT=true
OUTPUT_FILE=outputs/generated_layout.json
```

The CLI writes validated JSON. `outputs/` and traces are excluded from Git.

Local output is the safest development default. The API returns the layout in its response and does not use `OUTPUT_FILE` as a persistent API store. Cloud Run container storage is ephemeral, so use GCS or the response body when an artifact must survive a revision restart.

10.2 GCS

```
LOCAL_OUTPUT=false
GCS_BUCKET=approved-bucket-name
GCS_OBJECT_NAME=layouts/generated_layout.json
```

Success: CLI prints `Validated JSON saved (gcs): gs://...`. On failure check bucket, project, IAM, and object path. This is unrelated to checked-in baseline/special output folders.

10.3 Publisher handoff

Publisher handoff is the optional outbound POST to the external queue configured by `PUBLISHER_API_URL`. `OutputService` wraps validated JSON as `var documentData=<json>;` in the publisher envelope. The repository does not include the external dashboard, job

polling, or PDF retrieval workflow.

```
PUBLISHER_API_URL=https://approved-publisher-host/path
PUBLISHER_API_KEY=[secure value; never committed]
PUBLISHER_CREATED_BY=approved-operator-identity
PUBLISHER_PROJECT=approved-project-label
```

CLI requires `--publish`; API requires `"publish":true`. Keep false for smoke tests. Publisher failure returns 502 with `status=publish_failed` and preserves the validated layout.

A publisher smoke test is not equivalent to generation verification. It creates an outbound request that may create a real queued job. Before enabling it, identify the approved publisher environment, expected created-by/project metadata, who will confirm the queue entry, and how duplicate submissions are handled. If the API reports `publish_failed`, do not blindly resubmit because the remote system may have accepted the first request before the response failed.

11. Cloud Run deployment

11.1 Preflight

1. Use a reviewed commit in the repository root. Record `git rev-parse HEAD`.
2. Run Section 9 checks.
3. Require a clean `git status --short`. Do not deploy an uncommitted working tree.
4. Confirm `.gcloudignore` excludes secrets, `.env`, keys, archives, venv, logs, outputs, docs/tests, and raw data except font list.
5. Compare the candidate against the approved baseline with `git diff --name-status 3ceb766..HEAD`. For this fix, runtime changes must be limited to `api.py`, `streamlit_app.py`, and their focused tests.
6. Record the source commit in the release record. Do not add or change a Cloud Run environment variable merely for documentation.

11.2 Zero-traffic candidate

```
$PROJECT_ID="angular-lambda-421320"
$REGION="us-central1"
$SERVICE="layout-test-api"
$SUFFIX="candidate-YYYYMMDD"
gcloud.cmd run deploy $SERVICE --source . `
  --project=$PROJECT_ID --region=$REGION `
  --revision-suffix=$SUFFIX --tag=$SUFFIX --no-traffic `
  --quiet
```

Success: build SUCCESS, revision Ready, candidate URL printed, and stable traffic remains unchanged. Updating the existing service inherits configuration. For a new service/project, follow `GCP_SETUP.md` for APIs, runtime identity, pinned secrets, CPU/memory/concurrency/timeout/scale.

11.3 Candidate verification

1. GET `/health` without key: 200 and status ok. Use this path for external Cloud Run checks.
2. GET `/v1/templates` with key from Secret Manager: 200 and four templates.
3. POST fully resolved `publish=false`: 200, completed, non-empty validated layout, publisher null.
4. Confirm missing information uses **Submit details**, not Continue.
5. Run the deterministic failed-first/success-second UI check: require structured generation failure, Continue, a fresh request ID, and successful validated output with the saved answers.
6. Filter logs by request ID; expect retrieval, reranking, generation, validation, pipeline, request.
7. Compare candidate CPU, memory, service account, environment variables, and secret mappings with the approved baseline revision before promotion.

Cloud Run health-path note: the application defines both `/health` and `/healthz`, and both pass locally. In the verified Cloud Run environment, Google Front End returned its own HTML 404 for the exact external `/healthz` path while `/health`, `/docs`, and `/openapi.json` reached the application. No `/healthz` request appeared in container logs. Use `/health` as the documented external probe.

11.4 Promote and verify stable URL

```
gcloud.cmd run services update-traffic layout-test-api `
--project=angular-lambda-421320 --region=us-central1 `
--to-revisions=layout-test-api-$SUFFIX=100
```

Repeat health, templates, and `publish=false` generation on the stable URL. Record revision, request ID, timings, and rollback target.

Cloud Run deployment and traffic promotion

```
PS> gcloud.cmd run deploy layout-test-api --source . --no-traffic ...
Build: f338e486-17e0-4426-852b-b2d8fb015d26 SUCCESS
Revision: layout-test-api-continue-20260916
Candidate URL: https://continue-20260916---layout-test-api-6euw7j1ffa-uc.a.run.app
Initial traffic: 0%

PS> gcloud.cmd run services update-traffic layout-test-api `
--to-revisions=layout-test-api-continue-20260916=100
Routing traffic ... done
Stable URL: https://layout-test-api-6euw7j1ffa-uc.a.run.app
Active revision traffic: 100%
```

Verified zero-traffic deployment and 100% promotion.

Stable API end-to-end verification (publish=false)

```
GET /health -> 200 {"status":"ok"}
GET /v1/templates + x-api-key -> 200, 4 templates
POST /v1/layouts -> 200

{
  "status": "completed",
  "request_id": "stable_verify_20260916",
  "pages": 1,
  "publisher": null
}

Success: response status is completed, layout passed server validation,
publisher is null because publish=false, and the request returned through the stable URL.
```

Verified stable API end-to-end success with publishing disabled.

12. HTTP 502 investigation and resolution

12.1 Evidence and cause

Cloud Run request logs contained three representative 502 responses on 28 August 2026 against `layout-test-api-constraints-20260824`, after 98-251 seconds. Request `req_6d1f20f4077949b392a61a2fb541a3c7` logged three Gemini generations, three validation

retries, then `pipeline failed retries=2`. No publisher stage ran. The reported failures were therefore stochastic Gemini JSON/schema/geometry failures before publisher handoff, not a Cloud Run network gateway fault.

The approved production baseline was revision `layout-test-api-constraints-20260824`, verified against Git commit `3ceb766`. A later, broader revision was temporarily promoted during documentation work; it was returned to 0% and the approved baseline was restored to 100% before this focused fix was prepared.

12.2 Applied fix and verification

1. Kept the complete backend at the approved `3ceb766` baseline.
2. Changed exhausted generation failures from a generic HTTP detail response to structured JSON containing `status=generation_failed`, `retryable=true`, the request ID, and the error description.
3. Changed Streamlit to retain the prompt, template, accumulated answers, and publish choice after only that response.
4. Reserved **Continue** for this retryable generation-failure state. Missing required fields now use **Submit details**.
5. Continue resubmits the saved input with no request ID, which makes the API create a fresh request and run the full pipeline again.
6. Kept publisher failure separate; it preserves validated JSON but does not offer an automatic Continue because the remote queue may already have accepted the job.
7. Ran 58 tests. A deterministic browser scenario returned `needs_input`, then a retryable 502, then HTTP 200 completed; it asserted that the failed request carried all answers and the Continue request carried the same answers with `request_id=null`.
8. Confirmed with `git diff --name-only 3ceb766` that no semantic-planning, generation, retrieval, validation, dependency, Docker, or runtime-configuration implementation was added.

Status: the requested Continue recovery is implemented, verified locally, deployed as `layout-test-api-continue-20260916`, and receiving 100% traffic. Candidate and stable checks passed: health 200, structured generation-failure schema present, four templates, and a real `publish=false` generation completed with one validated page and `publisher=null`. The rollback target is `layout-test-api-constraints-20260824`.

```
HTTP 502 investigation and Continue recovery

Observed active revision: layout-test-api-constraints-20260824
Three client requests on 2026-08-28 returned HTTP 502 after 98-251 seconds.
Each log sequence ended with: stage=pipeline result=failed retries=2
No publisher stage ran: the failure occurred before publisher handoff.

Root cause: stochastic Gemini output failed JSON/schema/geometry validation on all 3 attempts.
Approved backend baseline: Git 3ceb766 / Cloud Run constraints-20260824.
No semantic, generation, retrieval, validation, dependency, or Docker change was added.

Corrected recovery behavior:
  API returns status=generation_failed, retryable=true, and the failed request ID.
  Layout Generator retains prompt, template, answers, and publish choice.
  Continue sends the same input with request_id=null to create a fresh request.
  Missing-information workflow uses Submit details, not Continue.
  Publisher failure is not automatically retried.

Verification: 58 tests passed. A deterministic browser run asserted 200 needs_input,
then 502 generation_failed, then a fresh 200 completed request with the same answers.
Deployment: continue-20260916 passed candidate and stable checks and has 100% traffic.
```

Redacted root-cause evidence, fixes, and verification.

12.3 Troubleshooting

Symptom	Action
Google HTML 404 for <code>/healthz</code> with no request log	Use <code>/health</code> for the external Cloud Run probe; verify <code>/docs</code> or <code>/openapi.json</code> if routing is still uncertain.

Application JSON 404	Verify the requested application path against <code>/openapi.json</code> .
401	Verify <code>x-api-key</code> , secure local variable, and active secret mapping.
422	Read detail: strict type/extra field/unknown template or answer/unsafe image URL.
502 <code>generation_failed</code>	Record the failed request ID. In Layout Generator, select Continue once to create a fresh request with the retained input. If it fails repeatedly, stop and inspect logs rather than looping indefinitely.
503	Verify revision environment variables, secret mappings, IAM, publisher configuration, and Vertex availability. Correct configuration before retrying.
502 <code>publish_failed</code>	Preserve layout/request ID; do not blindly retry because the queue may have accepted it.
Long request	Check the 900-second client/service timeout and filter logs by request ID to identify the slow stage.

```
gcloud.cmd logging read `
  'resource.type="cloud_run_revision" AND resource.labels.service_name="layout-test-api" AND jsonPayload.request_id="<REQUEST_ID>"' `
  --project=angular-lambda-421320 --limit=50 `
  --
  format='table(timestamp,resource.labels.revision_name,jsonPayload.stage,jsonPayload.result,jsonPayload.retries,jsonPayload.duration_ms)'
```

13. Security

- Never put live keys, service-account private keys, passwords, or tokens in Git, screenshots, documents, URLs, or shared Postman values.
- Use separate Secret Manager secrets for inbound and publisher keys; pin numeric versions.
- Use ADC locally and Cloud Run identity in deployment; do not package service-account JSON.
- Keep `publish=false` until a real publisher job is authorized.
- Use only public HTTPS image URLs.
- The client review PDF contained an API-key value. Treat it as exposed and rotate it in a coordinated change. It was not rotated here because that would invalidate existing client configuration.

14. Guided end-to-end exercise

1 - Access and branch

```
git clone https://github.com/jawadengineer12/gcp-vertex-ai.git
cd gcp-vertex-ai
git switch -c exercise/add-one-example
```

Success: expected files exist and current branch matches.

2 - Environment/authentication

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
uv sync --frozen
gcloud.cmd auth application-default login
gcloud.cmd config set project angular-lambda-421320
```

Success: venv Python is active, sync exits 0, project is correct.

3 - Add exactly one example

Create `raw_data/exercise_one_example.xlsx` with one valid row and exact required columns.

```
$env:EXCEL_FILE_PATH="raw_data/exercise_one_example.xlsx"
python -m scripts.append_excel_to_prompt_library
```

Success: appended 1, invalid 0, empty 0, total +1. Stop if different.

4 - Normalize and validate

```
python tests/validate_prompt_library.py
python -m scripts.normalize_prompt_library --source normalized_data/layout_prompt_library_updated.json --output
normalized_data/layout_prompt_library_sanitized.json
```

Success: new total validates and same number is normalized; review Git diff.

5 - Regenerate embeddings

```
python -m scripts.prepare_vertex_vector_data
python -m unittest tests.test_layout_pipeline.LocalVectorStoreTests
```

Success: vector count equals corpus, 3 tests OK.

6 - Tests

```
python -m unittest discover -s tests
python tests/validate_prompt_library.py
git diff --check
```

Success: 58 or current higher count OK, prompt validation passes, no diff errors.

7 - Generation and JSON

```
python main.py
```

Use relevant template, representative prompt, no `--publish`. **Success:** validated save message; run Section 6 direct schema check on saved file and require exit 0.

8 - Test revision

```
$SUFFIX="exercise-YYYYMMDD"
gcloud.cmd run deploy layout-test-api --source . --project=angular-lambda-421320 --region=us-central1 --revision-suffix=$SUFFIX --
tag=$SUFFIX --no-traffic --quiet
```

Success: build SUCCESS, revision Ready, tagged URL, 0% stable traffic.

9 API and Layout Generator verification

1. Use candidate URL.
2. Load the inbound key from Secret Manager into a secure local variable without printing it.
3. Verify `/health` and four templates.

4. Confirm an incomplete request shows **Submit details** and not Continue.
5. Use the deterministic retry check to force a generation failure after the available attempts.
6. Require the friendly warning and Continue; select it once.
7. Require a fresh request ID, completed status, non-empty validated JSON, publisher null, and Download JSON.
8. Filter logs by both request IDs.

Success: Continue appears only for the retryable generation failure; the second request reuses all saved input, succeeds, and removes the retry panel.

10 Handoff

```
git status --short
git diff --check
git add <intentional files>
git commit -m "docs: complete Phase 1 handoff"
git push -u origin <branch>
```

Commit the prompt library and vector file together with intentional code and tests. Do not add local archives, credentials, generated output, or unrelated user files. Record the source commit, revision, request ID, and rollback target. Stable promotion follows the normal team approval process.

15 Phase 1 release checklist

Gate	Required evidence
Source control	Focused branch pushed, working tree clean except known local-only files, source commit recorded
Baseline scope	<code>git diff --name-status 3ceb766..HEAD</code> contains only the approved retry behavior, focused tests, and documentation
Dependencies	<code>uv lock --check</code> succeeds
Unit tests	58 or current higher count reports OK
Corpus	Prompt library validates and vector count matches corpus count
Representative generation	Reviewed prompt completes with publishing disabled and saved JSON passes schema validation
Retry recovery	Forced generation failure shows Continue only after the structured 5xx; the fresh request keeps all input and can complete
Candidate revision	Ready at zero stable traffic and tagged URL passes health, templates, generation, and retry recovery
Stable revision	100% traffic, <code>/health</code> returns 200, API and Layout Generator pass
Security	No secret value appears in Git, screenshots, commands, logs, or the manual
Documentation	Manual, feedback matrix, screenshots, revision, and unresolved external checks agree

15.1 Items outside repository verification

- The client organization must identify the permanent GitHub access owner and confirm Henry's account onboarding if the repository becomes private.
- The client IAM administrator must define the exact human deployer grants.
- The external publisher owner must define queue inspection, job-status polling, generated-PDF retrieval, and duplicate-submission handling.
- The inbound API key exposed in the review PDF requires a coordinated rotation. Do not rotate it without updating every approved client.

Appendix A - Command reference

Task	Command
Venv	<code>python -m venv .venv</code>
Activate	<code>.\.venv\Scripts\Activate.ps1</code>
Install	<code>uv sync --frozen</code>
ADC	<code>gcloud.cmd auth application-default login</code>
CLI	<code>python main.py</code>
API	<code>uv run uvicorn api:app --host 0.0.0.0 --port 8080 --workers 1</code>
Streamlit	<code>uv run streamlit run streamlit_app.py</code>
Append	<code>python -m scripts.append_excel_to_prompt_library</code>
Normal normalize	<code>python -m scripts.normalize_prompt_library --source ... --output ...</code>
Special normalize	<code>python -m scripts.normalize_special_case_library --source ... --output ...</code>
Embeddings	<code>python -m scripts.prepare_vertex_vector_data</code>
Tests	<code>python -m unittest discover -s tests</code>
Special-case tests	<code>python -m unittest tests.test_special_cases.test_special_cases</code>
Live generation	<code>python main.py</code> with a reviewed prompt and no <code>--publish</code>

Appendix B - Glossary

Term	Meaning
Baseline outputs	Checked-in normal regression JSON under <code>baseline_outputs/</code> .
Branch	Named Git line of work.
Deployment revision	Immutable Cloud Run version to which traffic can be routed.
Embedding	768-dimensional retrieval vector; not a fine-tuned model.
GCS output	Optional validated-JSON upload to configured Cloud Storage.
Publisher handoff	Explicit outbound submission to external publisher queue.
Special case	Relationship-aware whole layout such as threaded frames or image spread.
Validation success	Schema and all production rules passed; API completed or CLI validated save.

End of Phase 1 final manual. Phase 2 is intentionally excluded.